# CHAPTER 3 Approximations and Round-off Errors

- we want to limit errors in our numerical solutions, since they involve approximations.

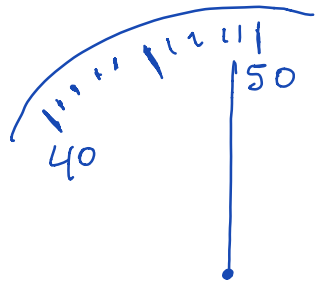
Numerical errors:

Round-off error: computers can only represent numbers with a finite number of digits.

Truncation error: introduced by the numerical methods which use approximations to represent exact mathematical operations.

## Significant Figures

EX Speedometer




$\Rightarrow$ 49 km/hr approx.

- we want to know the speed to one decimal place. one may say 48.8, another 48.9
- Limitations of the instrument $\Rightarrow$ only the first two digits may be used with confidence

Significant digits: of a number are those that can be used with confidence. They correspond to the # of certain digits plus one estimated digit.

- Speedometer : two certain digits 48. It is conventional to set the estimated digit at one half of the smallest scale division on the measurement device

  Three significant figures : 48.5

- Zeros are not always significant figures.

  0.00001845
  0.0001845
  0.001845

  } all have 4 significant figures

  ↳ zeros are locating the decimal pt.

- In large number, it may not be clear how many zeros are significant.

  Ex 45,300 may have 3, 4, or 5 significant digits, if zeros are known with confidence.

- Resolve uncertainty by using scientific notation

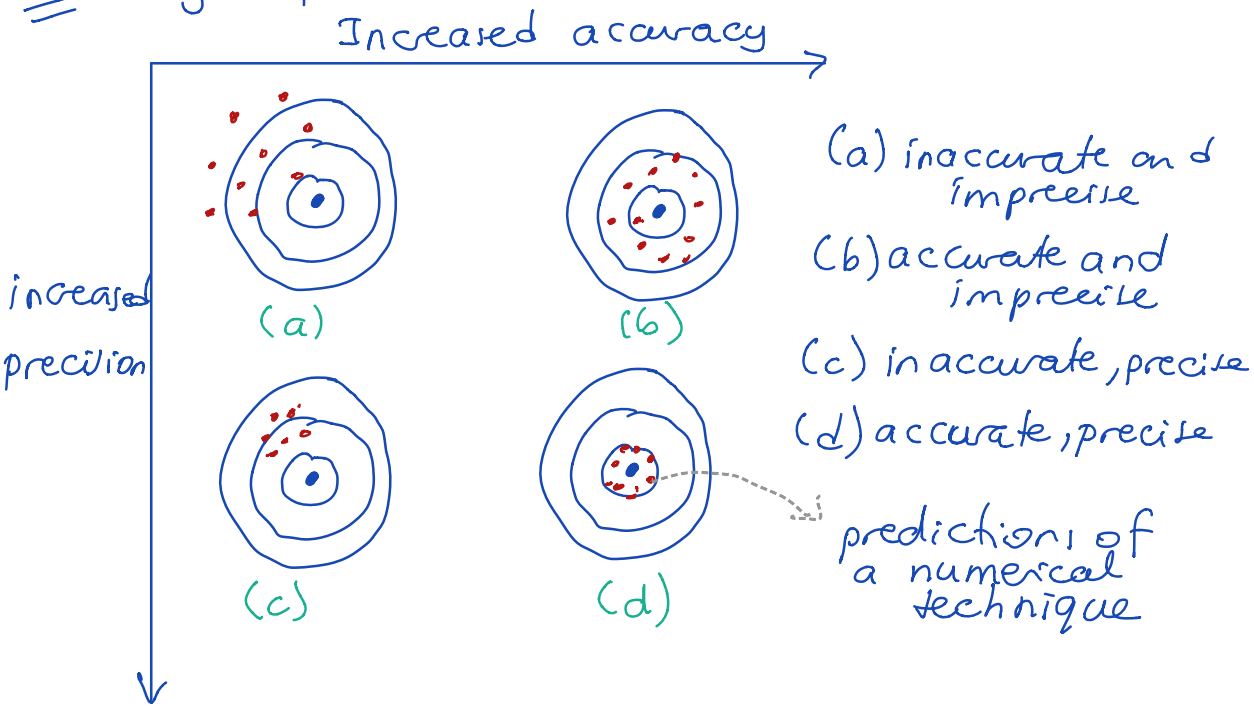
  $4.53 \times 10^4$ → 3 significant figures
  $4.530 \times 10^4$ → 4 " "
  $4.5300 \times 10^4$ → 5 " "

⇒ Importance : You may a numerical solution if it is correct & e.g. 4 significant figures.

Accuracy: How closely a computed/measured value agrees with the true value.

Precision: How closely individual computed/measure values agree with each other.

Ex Target practice




- We want accurate and precise numerical solutions.

error: represents both the inaccuracy and imprecision.

Error Definitions

Truncation errors: result when approximations are used to represent exact mathematical procedures.

Round-off errors: when numbers having limited significant figures are used to represent exact numbers.

True value = approximation + error

$E_t$ = true value − approximation

$E_t$: true error

We can normalize the error using the true value to account for the magnitudes of quantities:

True fractional relative error = $\frac{\text{true error}}{\text{true value}}$

$$\varepsilon_t = \frac{\text{true error}}{\text{true value}} \cdot 100\,\%$$

↳ true percent relative error

Ex Tasks of measuring the lengths of a bridge and a rivet.

| | True values | Measured values |
|---|---|---|
| bridge: | 10,000cm | 9999cm |
| rivet: | 10cm | 9cm |

Compute a) True error b) true percent relative error

a) $E_t = 10000 - 9999 = 1\text{cm}$ (bridge)

$E_t = 10 - 9 = 1\text{cm}$ (rivet)

b) $\varepsilon_t = \frac{1}{10000}\,100\% = 0.01\,\%$ (bridge)

$\varepsilon_t = \frac{1}{10}\,100\% = 10\,\%$ (rivet)

Conclusion: an adequate job of measuring the bridge, but not the rivet

In real life the true value is unknown.
Alternative method: normalize the error using the best available estimate of the true value:

$$\varepsilon_a = \frac{\text{approximate error}}{\text{approximation}} 100\%$$

approximate value

- In iterative approaches: successively compute better approximations.

Percent relative error:

$$\varepsilon_a = \frac{\text{current approximation} - \text{previous approximation}}{\text{current approximation}} 100\%$$

Repeat iterations until:

$$|\varepsilon_a| < \varepsilon_s$$ → prespecified percent tolerance

It can be shown that if $\varepsilon_s = (0.5 \times 10^{2-n})\%$ (1) then the result is correct to at least n significant digits.

Ex Error estimates for iterative methods

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots$$

(Maclaurin Series expansion)

Estimate $e^{0.5}$ by adding terms until $\varepsilon_a$ is less than $\varepsilon_s$, conforming to 3 significant digits.

Soln: Use (1) $\varepsilon_s = (0.5 \times 10^{2-3})\% = 0.05\%$

True value: $e^{0.5} = 1.648721\ldots$

First estimate $e^{0.5} \simeq 1$

Second estimate $e^{0.5} \simeq 1 + 0.5 = 1.5$

$$\varepsilon_t = \frac{1.648721 - 1.5}{1.648721} 100\% = 9.02\%$$

$$\varepsilon_a = \frac{1.5 - 1}{1.5} 100\% = 33.3\% \quad \text{not less than } \varepsilon_s$$

Add terms

After 6 iterations the result is accurate to 5 digits

use 6 terms: $e^{0.5} \simeq 1.648697$

$$\varepsilon_t = 0.00142\%$$

$$\varepsilon_a = 0.0158\% < \varepsilon_s = 0.05\%$$

## Round-off Errors

- Numbers such as $\pi, e, \sqrt{7}$ can't be represented exactly by a computer.

### Computer Representation of Numbers:

binary digits = bits

words: consist of bits

Numbers are typically stored in one or more words.

## Number Systems:

### Decimal, base-10 system: 10 digits

$8\ 6\ 4\ 0\ 9 = (8\times10^4)+(6\times10^3)+(4\times10^2)+(0\times10^1)+(9\times10^0)$

$10^4\ 10^3\ 10^2\ 10^1\ 10^0$

### Binary or base-2 systems:

$11 = (1\times2^1)+(1\times2^0) = 2+1 = 3$

$(1\ 0\ 1\ 0\ 1\ 1\ 0\ 1)_2$

$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$= (1\times128)+(0\times64)+\cdots+(1\times2^0) = (173)_{10}$

### Integer Representation

#### Signed Magnitude Method

→ employ the first bit for the sign: 0 for positive, 1 for negative

→ Remaining bits are used for storing the number

Ex 16-bit computer; representation of −173

1 0 0 0 0 0 0 0 1 0 1 0 1 1 0 1

sign — number (15 bits)

word = 16-bits

Ex Range of integers

Determine the range of integers in base-10 that can be represented on a 16-bit computer.

Sol$^n$ : First bit holds the sign. The other 15 bits from 0 to 111...111 (15 bits)

upper limit

$$(1\times 2^{14}) + (1\times 2^{13}) + \cdots + (1\times 2^{0}) = 32767 = (2^{15}-1)$$

Range is : $-32767$ to $32767$

it is redundant to use $1000\ldots000$ (15 bits) "minus zero"

If is employed to represent an additional negative number :

$-32768$ to $32767$

- Using 32 bits, largest integer $+2147483647$
- Signed magnitude method is not used to represent integers on conventional computers.
- 2's complement technique directly incorporates the sign into the number's magnitude rather than providing a separate bit to represent plus or minus.

# Fixed-point versus Floating Point representation

- Most scientific computations are performed using real numbers, that is, numbers with a fractional part.
- Question 1: how many bits are needed to encode real number?
- Question 2: how to represent real numbers using these bits?
- In engineering normally at least 7 to 8 significant digits precision is required.
  - Hence number of bits to encode 8 decimal digits is approximately 26 (since $\log_2 10 = 3.32$ bits are needed on the average to encode a digit)
- In computers, numbers are stored as a sequence of 8-bit bytes.
- Thus for 26 bits, 32 bits (4 bytes) which is bigger than 26 bits is a logical size to use for real numbers.
- How do we break the 32 bits into an integer part and a fractional part?

One method is shown below:

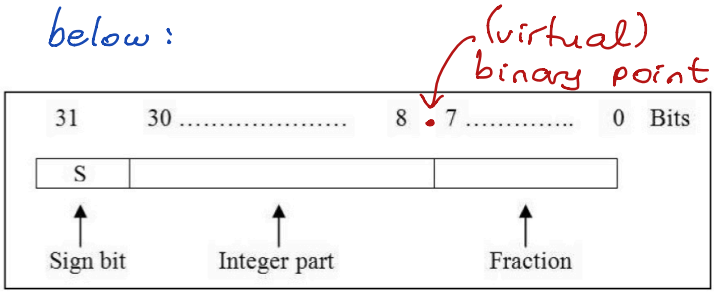



**Figure 1.** Fixed point representation of real numbers in binary using 32 bits.

With this fixed-pt representation, the largest and smallest positive binary numbers that may be represented using 32 bits are:

Largest: + 111 ..... 1.11111111 = 1677215.998046875
(23 bits, 8 bits)

Smallest: + 000 ... 0.00000001 = 0.00390625
(23 bits, 8 bits)

} range is small

- This range of real numbers, when fixed-pt representation is used, is not sufficient in many practical problems.
- Another representation called normalized floating point representation is used for real numbers in computers.
  - → 32 bits are divided into two parts:
    - → Mantissa with its sign (represents fractions)
    - → Exponent with its sign (power of 2 which multiplies the mantissa)

$$(\text{sign}) \times \text{mantissa} \times 2^{\pm \text{exponent}}$$

(sign): 1 bit; mantissa: binary fraction with a non-zero leading bit; exponent: binary integer

Q1. How many bits are to be used for the mantissa? (reserve 1 bit for the sign of the number)

Q2. How many bits are to be used for the exponent?

Q3. How to represent the sign of the exponent?

Answer Q1: to achieve at least 7 significant decimal digits, approx. 23 bits are needed for the mantissa

Answer Q2: ⇒ 8 bits for the exponent.

Answer Q3: Exponent may be represented in sign magnitude form. ⇒ Range of exponent: −127 to 127

Disadvantage: two representations for 0 exponent: +0 and −0

To avoid this one may use biased format to represent the exponent, where the exponent has no sign.

The range 0 to 255 of an 8-bit number is divided into two parts: 0 – 127 and 128 – 255

Exponent bits: 0 (negative) 127 (positive) 255

$\Rightarrow$ Range of exponent $-127$ to $+128$.

The largest floating pt. numbers using 32 bits:

$0.\underbrace{111\ldots111}_{23\text{ bits}} \times 2^{11111111}$

$\cong 3.4 \times 10^{38}$

Smallest floating pt. number is:

$0.\underbrace{100\ldots00}_{23\text{ bits}} \times 2^{-127} \cong 0.293 \times 10^{-38}$

IEEE 754 Standard uses the most significant bit of mantissa as always 1, on the left of the (virtual) decimal point.

$(-1)^S \times (1.f)_2 \times 2^{\text{exponent}-127}$

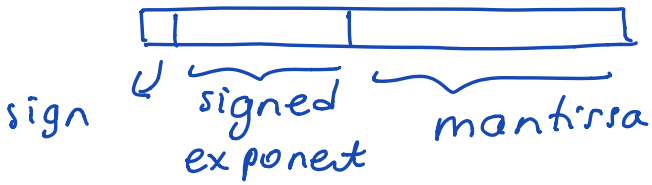
## Floating-point Representation

- Fractional quantities will be represented in floating-pt. form.
  - fractional part (mantissa or significand)
  - integer part (exponent or characteristic)

e → exponent

mb → base of the number system used
↳ mantissa

Ex 156.78 $\longrightarrow 0.15678 \times 10^{3}$ in floating point base-10 system.




- mantissa is <u>normalized</u> if it has leading zero digits.

Ex $\frac{1}{34} = 0.02941176\ldots$

- Floating pt base-10 system with 4 decimal places: $0.0294 \times 10^{0}$
- normalize to drop the leading zero:

$0.2941 \times 10^{-1}$

↳ retain an additional significant figure.

- Due to normalization:

$\frac{1}{b} \leq m < 1$

b: base $\Rightarrow$ base 10: $0.1 \leq m < 1$

base 2: $0.5 \leq m < 1$

- <u>Advantage</u>: both fractions and very large numbers can be represented.

- Disadvantages: . take-up more room } than integers
  . takes longer to process }
  . mantissa has a finite # of significant figures ⇒ round-off error.

Ex Hypothetical set of floating-pt numbers
Store information using 7-bit words.
1st bit : sign of the number
2nd bit : " " exponent
3rd-4th bits : magnitude of the exponent
last three : " " mantissa

Smallest positive number:

| $2^1$ | $2^0$ | $2^{-1}$ | $2^{-2}$ | $2^{-3}$ | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 0 | 0 |

: $(0.100)_2 \times 2^{-(11)_2}$

sign of the number
magnitude of exponent
magnitude of mantissa
sign of exponent

exponent : $(1 \times 2^1) + (1 \times 2^0) = 3 \Rightarrow$ exponent $= -3$
mantissa : $(1 \times 2^{-1}) + (0 \times 2^{-2}) + (0 \times 2^{-3}) = 0.5$
Smallest positive number is : $0.5 \times 2^{-3} = (0.0625)_{10}$
Next highest number:
$0111101 = (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-3} = (0.078125)_{10}$
$0111110 = (1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}) \times 2^{-3} = (0.093750)_{10}$

$01\,11\,111 = (1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3})\times2^{-3} = (0.109375)_{10}$

spaced evenly with an interval of 0.015625

To continue increasing:

set exponent = 10 $\Rightarrow (1\times2^{1}) + (0\times2^{0}) = 2$

$\Rightarrow$ exponent $= -2$

set mantissa = 100

$01\,10\,100 = (0.125000)_{10}$

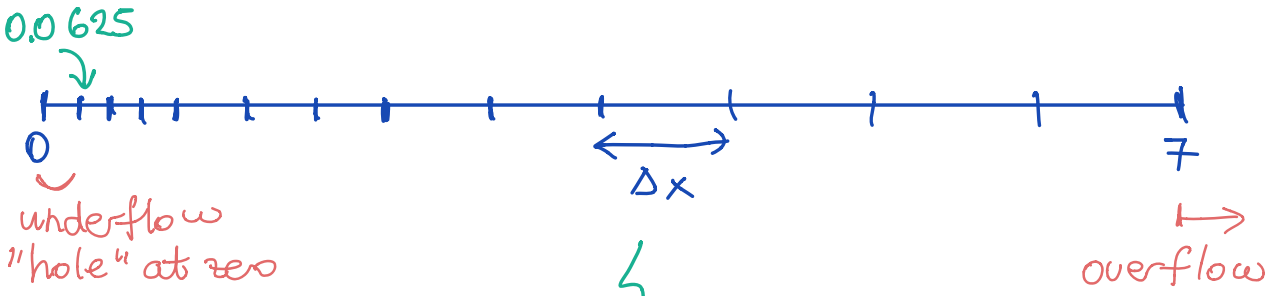
$01\,10\,101 = (0.156250)_{10}$ } 0.03125

$01\,10\,110 = (0.187500)_{10}$

$01\,10\,111 = (0.218750)_{10}$

Max. number

$0011111 = (7)_{10}$




the gap between numbers increase as the numbers grow in magnitude

**Machine Epsilon: (Machine Precision)**

There is a number $\varepsilon_m > 0$ such that

$1 + \delta = 1$, whenever $|\delta| < \varepsilon_m$ ↳ machine epsilon

**Ex** base $b = 2$

# of mantissa bits $t = 3$

$$\boxed{\varepsilon_m = b^{1-t}}$$

$$\varepsilon = 2^{1-3} = 2^{-2} = 0.25$$

---

**IEEE format:** 24 bits → mantissa, 7 significant digits

range ~ $10^{-38} - 10^{39}$

**Double precision:** 15-16 decimal digits of precision

range: $10^{-308} - 10^{308}$

## Arithmetic Manipulations of Computer Numbers

- Arithmetic operations can also cause round-off errors.
- Assume a hypothetical decimal computer with a 4-digit mantissa and a 1-digit exponent.
- we want to add: $0.1557 \times 10^{1}$ and $0.4381 \times 10^{-1}$

  $0.4381 \times 10^{-1} \longrightarrow 0.004381 \times 10^{1}$ (shift the second number)

Add:

$$\begin{array}{r} 0.1557 \times 10^{1} \\ +\ 0.004381 \times 10^{1} \\ \hline 0.160081 \times 10^{1} \end{array}$$

$\longrightarrow$ chop the result to $0.1600 \times 10^{1}$

(lost digits: 81)

- To subtract 26.86 from 36.41

$$\begin{array}{r} 0.3641 \times 10^{2} \\ -\ 0.2686 \times 10^{2} \\ \hline 0.0955 \times 10^{2} \end{array} \xrightarrow{\text{normalize}} 0.9550 \times 10^{1}$$

(0 not significant, added to fill the empty space)

- Subtraction when two numbers are very close

$$\begin{array}{r} 0.7642 \times 10^{3} \\ -\ 0.7641 \times 10^{3} \\ \hline 0.0001 \times 10^{3} \end{array} \Longrightarrow 0.1 \times 10^{0} = 0.1000$$

- three non-significant zeros are appended
- Subsequent manipulations will act as if these zeros are significant.
- Greatest source of error in numerical methods.

- Multiplication

$(0.1363 \times 10^{3}) \times (0.6423 \times 10^{-1}) = 0.08754549 \times 10^{2}$

(add the exponents and multiply the mantissas)

↓ normalize

$\Rightarrow 0.8754549 \times 10^{1}$

chop $\Rightarrow 0.8754 \times 10^{1}$

- Adding a large and a small number

Add: 0.0010 and 4000

$$\begin{array}{r} 0.4000 \times 10^{4} \\ + \; 0.0000001 \times 10^{4} \\ \hline 0.4000001 \times 10^{4} \end{array}$$

↓ chop

$0.4000 \times 10^{4}$

(a hypothetical computer with 4 digit mantissa and 1 digit exponent)

↓

no effect of addition.

- This type error can occur in the computation of infinite series. The initial terms are large as compared to later terms.

  Solution: add the small terms first, and then add the large numbers.

# Chapter 4 Truncation Errors and Taylor Series

## Taylor Series:

- Provides a mean of predicting a function value at one pt. i.t.o the function value and its derivative at another point.
- Thm states that any smooth function can be approximated as a polynomial.

### Taylor's Thm

$$f(x) = f(a) + f'(a)(x-a) + \frac{1}{2!}f''(a)(x-a)^2 + \frac{1}{3!}f'''(a)(x-a)^3 + \cdots$$

($f(x) \to x_{i+1}$, $f(a) \to x_i$)

$$+ \frac{1}{n!}f^{(n)}(a)(x-a)^n + R_n$$

$$R_n = \int_a^x \frac{(x-t)^n}{n!} f^{(n+1)}(t)\,dt \quad , t: \text{dummy variable}$$

Assumptions: $f(x)$ and its first $(n+1)$ derivatives are continuous on an interval containing a and x.

### Term by Term

first term $f(x_{i+1}) \cong f(x_i)$ → zero order approximation

second term $f(x_{i+1}) \cong f(x_i) + f'(x_i)(x_{i+1}-x_i)$ → first-order approximation

straight line

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)(x_{i+1}-x_i) + \frac{1}{2!}f''(x_i)(x_{i+1}-x_i)^2 + \cdots$$

$h$ (stepsize)

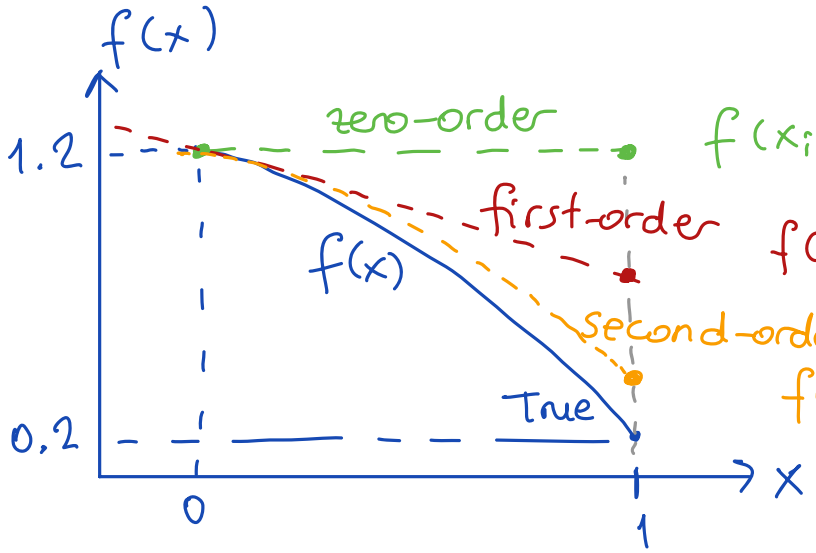
$$+ \frac{f^{(n)}(x_i)}{n!}(x_{i+1}-x_i)^n + R_n$$

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}\underbrace{(x_{i+1}-x_i)}_{h:\text{stepsize}}^{n+1} = O(h^{n+1})$$ , where $\xi$ is a pt. btwn $x_i$ and $x_{i+1}$

EX $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$

Use zero to 4th order Taylor series expansion to approximate the function.

$x_i = 0$ with $h = 1 \Rightarrow$ Predict $f(x)$ at $x = 1$




$f(x_{i+1}) \cong f(x_i) \Rightarrow f(1) = f(0)$

$f(x_{i+1}) \cong f(x_i) + f'(x_i)h$

$f(x_{i+1}) \cong f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2$

$f(0) = 1.2$ , $f(1) = 0.2$ ↝ True value we're trying to predict

| order | Approximation | Truncation Error at $x=1$ |
|---|---|---|
| $n=0$ | $f(1) \cong 1.2$ | $E_t = 0.2 - 1.2 = -1$ |
| $n=1$ | $f'(x)\Big\|_{x=0} = -(0.4)x^3 - 0.45x^2 - x - 0.25\Big\|_{x=0}$ | |
| | $f'(0) = -0.25$ | |
| | $f(1) \cong 1.2 - 0.25h$ | |
| | $f(1) \cong 0.95$ | $E_t = 0.2 - 0.95 = -0.75$ |
| $n=2$ | $f''(0) = -1$ | |
| | $f(x_{i+1}) \cong 1.2 - 0.25h - 0.5h^2$ | $E_t = 0.2 - 0.45 =$ |
| | $f(1) = 0.45$ | $-0.25$ |

* $R_n = O(h^{n+1})$ → error is proportional to $h^{n+1}$

e.g. if error is $O(h)$, halving the step size h, will halve the error.

if error is $O(h^2)$, halving the step size, error will quarter the error

## Numerical Differentiation

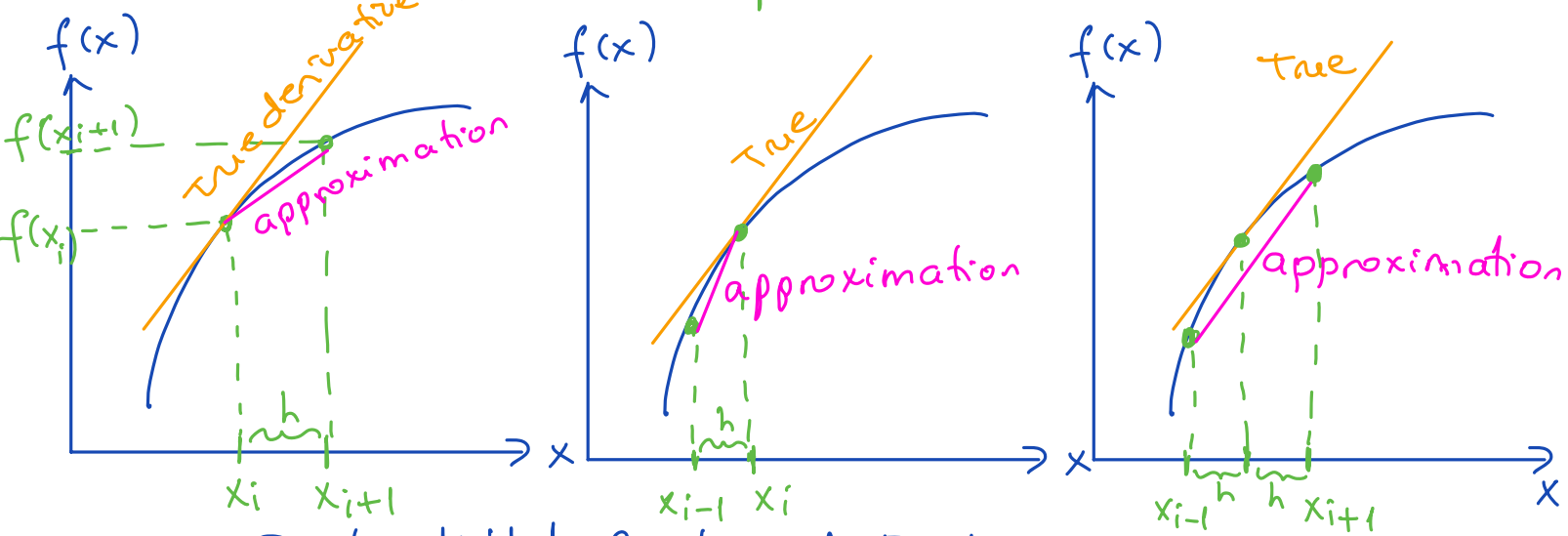
### Forward Finite Divided Difference

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} + O(\overbrace{x_{i+1} - x_i}^{h})$$

$$f'(x_i) = \frac{\Delta f_i}{h} + O(h)$$

$\Delta f_i$ → first forward difference

$h$ → step size




Forward Finite-divided Difference

Backward Finite-Divided Diff

Centered finite divided difference

### Backward Finite Divided Difference

$x_{i-1} - x_i = -h$

$$f(x_{i-1}) = f(x_i) - f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \cdots \quad (1)$$

Backward Taylor Series

$$f'(x_i) \cong \frac{f(x_i) - f(x_{i-1})}{h} + O(h)$$

Error is $O(h)$

# Centered Difference Approximation of the 1st order Derivative

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \dots \quad (2)$$

Subtract (1) from (2):

$$f(x_{i+1}) - f(x_{i-1}) = 2f'(x_i)h + \frac{2f^{(3)}(x_i)}{3!}h^3 + \dots$$

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} - O(h^2)$$

→ more accurate representation of the derivative

**Ex** Estimate the 1st derivative of (using forward, backward, centered methods)

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.25$$

at $x = 0.5$ using a step size of $h = 0.5$ and $h = 0.25$

True value: $f'(x) = -0.4x^3 - 0.45x^2 - x - 0.25$

$f'(0.5) = -0.925$

Approximations: $h = 0.5$ $\qquad x_{i+1} - x_i = h$, $x_i - x_{i-1} = h$

$x_{i-1} = 0$, $f(x_{i-1}) = 1.25$

$x_i = 0.5$, $f(x_i) = 0.925$

$x_{i+1} = 1$, $f(x_{i+1}) = 0.25$

Forward diff

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} = \frac{f(1) - f(0.5)}{1 - 0.5} = \frac{0.25 - 0.925}{0.5} = -1.45$$

$|\varepsilon_t| = 58.9\%$

Backward Diff

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}} = \frac{f(0.5) - f(0)}{0.5 - 0} = \frac{0.925 - 1.25}{0.5} = -0.55$$

$|\varepsilon_t| = 39.7\%$

Centered Diff

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{x_{i+1} - x_{i-1}} = \frac{f(1) - f(0)}{1} = \frac{0.2 - 1.25}{1} = -1, \quad |\varepsilon_t| = 9.6\%$$

$h = 0.25$

$x_{i-1} = 0.25$, $f(x_{i-1}) = 1.103$

$x_i = 0.5$, $f(x_i) = 0.925$

$x_{i+1} = 0.75$, $f(x_{i+1}) = 0.6363$

Forward

$$f'(0.5) = \frac{f(0.75) - f(0.25)}{0.25} = -1.155 \Rightarrow |\varepsilon_t| = 26.5\%$$

(Decreased by a factor of 2)

Backward

$$f'(0.5) = \frac{f(0.5) - f(0.25)}{0.25} = -0.714 \Rightarrow |\varepsilon_t| = 21.7\%$$

Centered

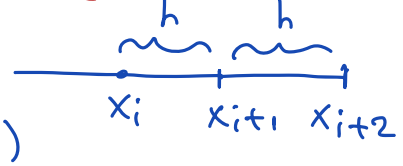
$$f'(0.5) = \frac{f(0.75) - f(0.25)}{0.5} = -0.934 \Rightarrow |\varepsilon_t| = 2.4\%$$

Decreased by a factor of 4.

more accurate than forward & backward.

Finite Order Approximations of Higher Derivatives

Forward Taylor series:




$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \cdots \quad (1)$$

Write a forward Taylor series expansion for $f(x_{i+2})$ i.t.o $f(x_i)$

$$f(x_{i+2}) = f(x_i) + f'(x_i)2h + \frac{f''(x_i)}{2!}(2h)^2 + \cdots \quad (2)$$

Multiply (1) by 2 and subtract from (2):

$$f(x_{i+2}) - 2f(x_{i+1}) = -f(x_i) + f''(x_i)h^2 + \cdots$$

$$f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2} + O(h)$$

Second forward Finite Difference

Second backward Finite Divided Difference

$$f''(x_i) = \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2})}{h^2} + O(h)$$

Centered version (more accurate)

$$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1})}{h^2} + O(h^2)$$

$$= \frac{\dfrac{f(x_{i+1}) - f(x_i)}{h} - \dfrac{f(x_i) - f(x_{i-1})}{h}}{h}$$

Backward for 1st deriv.

Derivative of a derivative

# Error Propagation

**Goal**: Study how errors in numbers can propagate through mathematical functions.

(e.g. if you multiply two numbers that have errors, estimate the error of the product)

## Functions of a single variable

function : $f(x)$

$\tilde{x}$ : an approximation of $x$

**Goal**: assess the effect of the discrepancy btwn $x$ and $\tilde{x}$ on the value of the function

$$\Delta f(\tilde{x}) = |f(x) - f(\tilde{x})|$$ ↳ can't compute this:

$f(x)$ is unknown because $x$ is unknown

**overcome this**: assume $\tilde{x}$ is close to $x$ and $f(\tilde{x})$ is continuous and differentible. Use Taylor series around $\tilde{x}$:

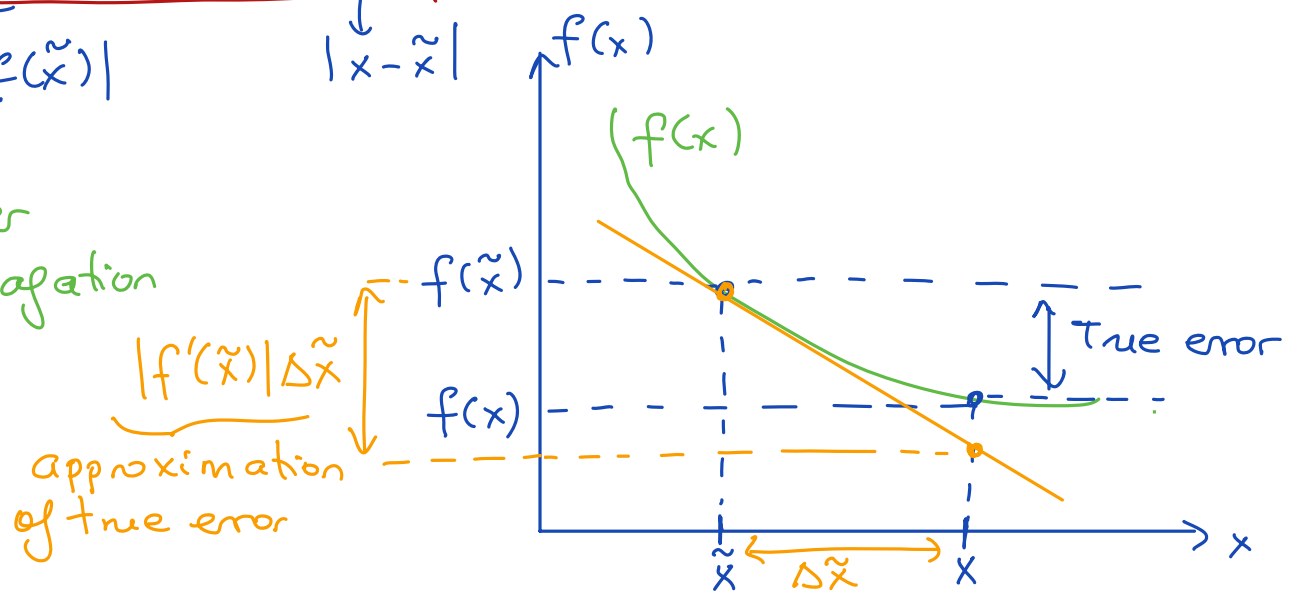
$$f(x) = f(\tilde{x}) + f'(\tilde{x})(x-\tilde{x}) + \underbrace{\frac{f''(\tilde{x})}{2!}(x-\tilde{x}) + \cdots}_{\text{Drop this}}$$

$$f(x) - f(\tilde{x}) = f'(\tilde{x})(x-\tilde{x})$$

$$\boxed{\Delta f(x) \cong |f'(\tilde{x})|\,\Delta\tilde{x}}$$

$\Delta f(x)$ : $|f(x) - f(\tilde{x})|$ ; $\Delta\tilde{x}$ : $|x - \tilde{x}|$

First order Error propagation



EX Given $\tilde{x} = 2.5$ with an error of $\Delta\tilde{x} = 0.01$, estimate the resulting error in the function $f(x) = x^3$.

$$\Delta f(\tilde{x}) \cong |f'(\tilde{x})| \Delta\tilde{x}$$

$$= 3\tilde{x}^2 \Big|_{\tilde{x}=2.5} \Delta\tilde{x} = 3(2.5)^2(0.01) = 0.1875$$

Since $f(2.5) = 15.625$ we can predict that

$$f(2.5) = 15.625 \pm 0.1875$$

or true value lies between 15.4375 and 15.8125

$$f(2.49) = 15.4375$$

$$f(2.51) = 15.8125$$

## Functions of More Than One Variable

Function of two independent variables $u$ and $v$, Taylor series can be written as:

$$f(u_{i+1}, v_{i+1}) = f(u_i, v_i) + \frac{\partial f}{\partial u}\Big|_{(u_i, v_i)} (u_{i+1} - u_i) + \frac{\partial f}{\partial v}\Big|_{(u_i, v_i)} (v_{i+1} - v_i)$$

$$+ \frac{1}{2!}\left[ \frac{\partial^2 f}{\partial u^2}(u_{i+1} - u_i)^2 + 2\frac{\partial^2 f}{\partial u \partial v}(u_{i+1} - u_i)(v_{i+1} - v_i) + \frac{\partial^2 f}{\partial v^2}(v_{i+1} - v_i)^2 \right] + \cdots$$

all partial derivatives are evaluated at the base pt. i.

- Drop 2nd order terms and solve

$$\Delta f(\tilde{u}, \tilde{v}) = \left|\frac{\partial f}{\partial u}\right| \Delta\tilde{u} + \left|\frac{\partial f}{\partial v}\right| \Delta\tilde{v}$$

$\Delta\tilde{u}, \Delta\tilde{v}$ : estimates of errors in $u$ and $v$

## Stability and Condition

- The condition of a mathematical problem relates to its sensitivity to changes in its input values.
- A computation is numerically unstable if the uncertainty in its values is grossly magnified by the numerical method.
  - Used a Taylor series

$$f(x) = f(\tilde{x}) + f'(\tilde{x})(x - \tilde{x})$$

Relative error of $f(x)$: $\dfrac{f(x) - f(\tilde{x})}{f(x)} \cong \dfrac{f'(\tilde{x})(x - \tilde{x})}{f(\tilde{x})}$

Relative error of $x$: $\dfrac{x - \tilde{x}}{\tilde{x}}$

$$\text{Condition number} = \frac{\text{Relative error of } f(x)}{\text{Relative error of } x} = \frac{\dfrac{f'(\tilde{x})(x - \tilde{x})}{f(\tilde{x})}}{\dfrac{x - \tilde{x}}{\tilde{x}}}$$

$$\text{Condition number (CN)} = \frac{\tilde{x}\, f'(\tilde{x})}{f(\tilde{x})}$$

- CN provides a measure of the extent to which the uncertainty in $x$ is magnified by $f(x)$.

if $CN = 1 \Rightarrow$ The function relative error is identical to the relative error in $x$

if $CN > 1 \Rightarrow$ relative error is magnified

if $CN \gg 1 \Rightarrow$ the function is ill-conditioned

Ex Compute and interpret the condition number of

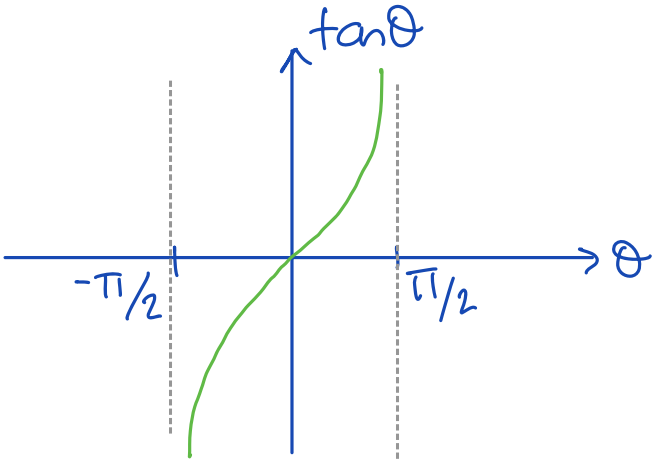
$f(x) = \tan x$ for $\tilde{x} = \frac{\pi}{2} + 0.1\left(\frac{\pi}{2}\right)$

$f(x) = \tan x$ for $\tilde{x} = \frac{\pi}{2} + 0.01\left(\frac{\pi}{2}\right)$

$$\text{Condition Number} = \frac{\tilde{x} f'(\tilde{x})}{f(\tilde{x})} = \frac{\tilde{x} \frac{1}{\cos^2 \tilde{x}}}{\tan \tilde{x}}$$

put $\tilde{x} = \frac{\pi}{2} + 0.1\left(\frac{\pi}{2}\right) \Rightarrow CN = -11.2 \Rightarrow$ Function is ill-conditioned

put $\tilde{x} = \frac{\pi}{2} + 0.01\left(\frac{\pi}{2}\right) \Rightarrow CN = -101 \Rightarrow$ "




- Major cause of ill-conditioning is the derivate at $\theta = \pi/2$

# Examples

**Example 2.6** In IEEE double precision arithmetic the associative rule for summation and multiplication may be violated as follows. The computation of

$$1 + 10^{17} - 10^{17}$$

as

$$1 \oplus (10^{17} \ominus 10^{17})$$

gives the correct answer 1, while the result of the machine summation

$$(1 \oplus 10^{17}) \ominus 10^{17}$$

is 0.

Why?

**Example 2.6** In IEEE double precision arithmetic the associative rule for summation and multiplication may be violated as follows. The computation of

$$1 + 10^{17} - 10^{17}$$

as

$$1 \oplus (10^{17} \ominus 10^{17})$$

gives the correct answer 1, while the result of the machine summation

$$(1 \oplus 10^{17}) \ominus 10^{17}$$

is 0.

$$1 = 0.1 \times 10^{1} = 0.00000000000000001 \times 10^{18}$$

$$0.1 \text{ x } 10^{18}$$

---

$$0.1 \text{ x } 10^{18}$$

In turn, the computation of

$$10^{155}10^{155}10^{-250}$$

as

$$10^{155} \otimes (10^{155} \otimes 10^{-250})$$

will produce an accurate approximation to the correct answer $10^{60}$, while the attempt to compute

$$(10^{155} \otimes 10^{155}) \otimes 10^{-250}$$

will give an overflow.

One of the most dangerous operations during the computations is the subtraction of numbers that are very close to each other (although in most cases this is done exactly in machine arithmetic). In this case a *subtractive cancellation* may occur.

**Example 2.7** Consider the numbers

$$x = 0.123456789\xi, \ y = 0.123456789\eta,$$

which agree in their first 9 significant decimal digits. The result of the subtraction is

$$z = x - y = \pm 0.000000000|\xi - \eta|.$$

If the original $x$ and $y$ are subject to errors, likely to be at least of order $10^{-10}$, then the subtraction has brought these errors into prominence and $z$ may be dominated by error. If in particular the first 9 digits of $x$ and $y$ are true, and the digits $\xi, \eta$ are uncertain, then there may be no true digit in the result $z$.

# Solving some illustrative problems by MATLAB

The commands

```
>> x = [-1:-1:-18]'; x = 10.^x
```

will produce the 18–vector $x = [10^{-1}, 10^{-2}, \ldots, 10^{-18}]^\top$. With this vector we may compute the quantities

$$y(k) = \frac{(1 + x(k)) - 1}{x(k)}, \ k = 1, 2, \ldots, 18,$$

where $x(k)$ are the elements of $x$, from

```
>> y = ((1 + x) - 1)./x
```

The result is

```
y =
 1.00000000000000
 1.00000000000000
 0.99999999999989
 0.99999999999989
 1.00000000000655
 0.99999999991773
 1.00000000058387
 0.00000000392253
 1.00000008274037
 1.00000008274037
 1.00000008274037
 1.00008890058234
 0.99920072216264
 0.99920072216264
 1.11022302462516
                0
                0
                0
```

Since the exact result is 1, we see that for $k = 15$ there is a relative error of 11 percent while for $k \geq 16$ the result is totally wrong with relative error of 100 percent.

The computation of the vector of ones

$$y(k) = \frac{(1 + x(k))^2 - 1 - 2x(k)}{x^2(k)}$$

by the command

```
>> y = ((1 + x).^2 - 1 - 2*x)./x.^2
```

will give

```
y =
 1.00000000000002
 1.00000000000006
 0.99999999969742
 0.99999999171889
 1.00000139299870
 0.99992436739695
 1.01087806660425
-1.215...
 ...
```

We see that for $k = 7$ the result is computed by 11 percent relative error, while for $k \geq 8$ the result is completely useless (for $k = 8$ even the sign is wrong and the relative error is 221 percent).

**Example 2.8** If we compute

$$y = \frac{(1+x)^2 - (2x+1)}{x^2}, \ x \neq 0,$$

in the ANSI/IEEE 754-1985 Standard for

$$x = 10^{-1}, 10^{-2}, \ldots, 10^{-n}, \ldots,$$

we shall see that for $n \geq 7$ the computed result $\widehat{y}$ is far from the correct answer $y = 1$; even negative values for $\widehat{y}$ will appear. The reason for this phenomenon is that we have cancellation in the numerator for small $x$. For $n > 8$ the result of the subtraction is of order $\varepsilon \approx 10^{-16}$ or less and there is no correct digit in the computed numerator. To worsen the situation, this wrong result is divided by a denominator that is less than $10^{-16}$. This example is very instructive because the input of order $10^{-8}$ and an output 1 are by no means close to the boundaries of the standard range $[10^{-324}, 10^{308}]$ of $\mathbb{M}$.

Similarly, the computation of the quantities

$$y(k) = \frac{(1+x(k))^3 - 1 - 3x(k) - 3x^2(k)}{x^3(k)},$$

all equal to 1, by the corresponding command gives

```
y =
    1.00000000000035
    0.99999999991242
    0.99999950547556
    0.99987350828302
    1.10953927987086
-202.14...
  ...
```

The results are self-evident.

Otherwise speaking, *cancellation is the loss of true significant digits when small numbers are computed by subtraction from large numbers.*

Often subtractive cancellation can be avoided by a simple reformulation of the problem.

**Example 2.10** The expression

$$y = \sqrt{1+x} - 1$$

may be computed as

$$y = \frac{x}{1+\sqrt{1+x}}$$

to avoid cancellation for small $x$.

As a summary, one has to remember the following practical rules about machine arithmetic.

- *The rounding unit* $\mathbf{u}$ *is about* $10^{-16}$ *which means that the rounding and the arithmetic operations are done with 15-16 true significant decimal digits.*

- *For safe and accurate arithmetic calculations the (non-zero) operands and the result must have absolute values in the normal range between* $10^{-308}$ *and* $10^{308}$*. A tolerance of several orders of magnitude may increase the reliability of computations.*

- *Subtraction of close approximate numbers should be avoided whenever possible or at least be carefully watched.*

# Control of Numerical Errors - Guidelines

- Avoid subtracting two equal numbers, to prevent loss of significance
- Rearrange or reformulate the problem to avoid subtractive cancellation.
- When adding or subtracting numbers, sort the numbers and work with the smallest numbers first (to avoid loss of significance)
- Prediction of total numerical error is very complicated for even moderately sized problems.
- Repeat the computations with different parameters and compare the results.

# Control of Numerical Errors - Guidelines

- **Sensitivity analysis**: see how your solution changes when you change the model parameters or input values.
- Try different numerical algorithms (with different convergence and stability properties)
- Critical problems involving human life:
  - Let two or more independent groups solve it, and compare the results.

# Unavoidable Errors in Computing

Gerald W. Recktenwald
Department of Mechanical Engineering
Portland State University
gerry@me.pdx.edu

These slides are a supplement to the book *Numerical Methods with* MATLAB*: Implementations and Applications*, by Gerald W. Recktenwald, © 2000–2006, Prentice-Hall, Upper Saddle River, NJ. These slides are copyright © 2000–2006 Gerald W. Recktenwald. The PDF version of these slides may be downloaded or stored or printed for noncommercial, educational use. The repackaging or sale of these slides in any form, without written consent of the author, is prohibited.

The latest version of this PDF file, along with other supplemental material for the book, can be found at `www.prenhall.com/recktenwald` or `web.cecs.pdx.edu/~gerry/nmm/`.

Version 1.0 August 19, 2006

# Overview (1)

- Digital representation of numbers
  - ▷ Size limits
  - ▷ Resolution limits
  - ▷ The floating point number line
- Floating point arithmetic
  - ▷ roundoff
  - ▷ machine precision

# Overview (2)

- Implications for routine computation
  - ▷ Use "close enough" instead of "equals"
  - ▷ loss of significance for addition
  - ▷ catastrophic cancellation for subtraction
- Truncation error
  - ▷ Demonstrate with Taylor series
  - ▷ Order Notation

# What's going on here?

Spontaneous generation of an insignificant digit:

```
>> format long e  % display lots of digits
>> 2.6 + 0.2
ans =
        2.800000000000000e+00
>> ans + 0.2
ans =
        3.000000000000000e+00
>> ans + 0.2
ans =
        3.200000000000001e+00     Why does the least significant digit appear?
>> 2.6 + 0.6
ans =
        3.200000000000000e+00     Why does the small error not show up here?
```

# Bits, Bytes, and Words

| base 10 | conversion | base 2 |
|---|---|---|
| 1 | $1 = 2^0$ | 0000 0001 |
| 2 | $2 = 2^1$ | 0000 0010 |
| 4 | $4 = 2^2$ | 0000 0100 |
| 8 | $8 = 2^3$ | 0000 1000 |
| 9 | $8 + 1 = 2^3 + 2^0$ | 0000 1001 |
| 10 | $8 + 2 = 2^3 + 2^1$ | 0000 1010 |
| 27 | $16 + 8 + 2 + 1 = 2^4 + 2^3 + 2^1 + 2^0$ | $\underbrace{0001\ 1011}_{\text{one byte}}$ |

# Digital Storage of Integers (1)

As a prelude to discussing the binary storage of floating point values, first consider the binary storage of integers.

- Integers can be exactly represented by base 2
- Typical size is 16 bits
- $2^{16} = 65536$ is largest 16 bit integer
- $[-32768, 32767]$ is range of 16 bit integers in twos complement notation
- 32 bit and larger integers are available

# Digital Storage of Integers (2)

**Note:** Unless explicitly specified otherwise, all mathematical calculations in MATLAB use double precision floating point numbers.

**Expert's Note:** The built-in `int8`, `int16`, `int32`, `uint8`, `uint16`, and `uint32` classes are used to reduce memory usage for very large data sets.

# Digital Storage of Integers (3)

Let $b$ be a binary digit, i.e. 1 or 0

$$(bbbb)_2 \quad \Longleftrightarrow \quad |2^3|2^2|2^1|2^0|$$

The **rightmost bit** is the **least significant bit** (LSB)

The **leftmost bit** is the **most significant bit** (MSB)

**Example:**

$$(1001)_2 = 1 \times 2^3 \ + \ 0 \times 2^2 \ + \ 0 \times 2^1 \ + \ 1 \times 2^0$$

$$= 8 + 0 + 0 + 1 = 9$$

# Digital Storage of Integers (4)

**Limitations**:

- A *finite* number of bits are used to store each value in computer memory.

- *Limiting the number of bits limits the size* of integer that can be represented

| | | | |
|---|---|---|---|
| largest 3 bit integer: | $(111)_2$ | $= 4 + 2 + 1 = 7$ | $= 2^3 - 1$ |
| largest 4 bit integer: | $(1111)_2$ | $= 8 + 4 + 2 + 1 = 15$ | $= 2^4 - 1$ |
| largest 5 bit integer: | $(11111)_2$ | $= 16 + 8 + 4 + 2 + 1 = 31$ | $= 2^5 - 1$ |
| largest $n$ bit integer: | | | $= 2^n - 1$ |

# Digital Storage of Floating Point Numbers (1)

Numeric values with non-zero fractional parts are stored as **floating point numbers**.

All floating point values are represented with a normalized scientific notation[1].

**Example:**

$$12.2792 = \underbrace{0.123792}_{\text{Mantissa}} \times 10^{2} \quad \text{Exponent}$$

[1] The IEEE Standard on Floating Point arithmetic defines a normalized *binary* format. Here we use a simplified *decimal* (base ten) format that, while abusing the standard notation, expresses the essential ideas behind the decimal to binary conversion.

# Digital Storage of Floating Point Numbers (2)

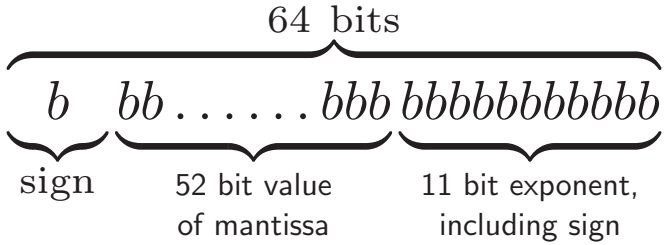
Floating point values have a fixed number of bits allocated for storage of the mantissa and a fixed number of bits allocated for storage of the exponent.

Two common precisions are provided in numeric computing languages

| Precision | Bits for mantissa | Bits for exponent |
|---|---|---|
| Single | 23 | 8 |
| Double | 53 | 11 |

# Digital Storage of Floating Point Numbers (3)

A double precision (64 bit) floating point number can be schematically represented as




The finite number of bits in the exponent limits the magnitude or *range* of the floating point numbers.

The finite number of bits in the mantissa limits the number of significant digits or the *precision* of the floating point numbers.

# Digital Storage of Floating Point Numbers (4)

The floating point mantissa is expressed in powers of $\frac{1}{2}$

$\left(\frac{1}{2}\right)^0 = 1$ is not used

$\left(\frac{1}{2}\right)^1 = 0.5 \qquad \left(\frac{1}{2}\right)^2 = 0.25 \qquad \left(\frac{1}{2}\right)^3 = 0.125 \qquad \cdots$

# Digital Storage of Floating Point Numbers (5)

## Algorithm 5.1 Convert Floating-Point to Binary

$r_0 = \mathrm{x}$
for $k = 1, 2, \ldots, m$
  if $r_{k-1} \geq 2^{-k}$
    $b_k = 1$
    $r_k = r_{k-1} - 2^{-k}$
  else
    $b_k = 0$
    $r_k = r_{k-1}$
  end if
end for

# Digital Storage of Floating Point Numbers (6)

**Example:** *Binary mantissa for* $x = 0.8125$ — *Apply Algorithm 5.1*

| $k$ | $2^{-k}$ | $b_k$ | $r_k = r_{k-1} - b_k 2^{-k}$ |
|---|---|---|---|
| 0 | NA | NA | 0.8125 |
| 1 | 0.5 | 1 | 0.3125 |
| 2 | 0.25 | 1 | 0.0625 |
| 3 | 0.125 | 0 | 0.0625 |
| 4 | 0.0625 | 1 | 0.0000 |

Therefore, the binary mantissa for 0.8125 is (exactly) $(1101)_2$

# Digital Storage of Floating Point Numbers (7)

**Example:** *Binary mantissa for* $x = 0.1$ — *Apply Algorithm 5.1*

| $k$ | $2^{-k}$ | $b_k$ | $r_k = r_{k-1} - b_k 2^{-k}$ |
|---|---|---|---|
| 0 | NA | NA | 0.1 |
| 1 | 0.5 | 0 | 0.1 |
| 2 | 0.25 | 0 | 0.1 |
| 3 | 0.125 | 0 | 0.1 |
| 4 | 0.0625 | 1 | 0.1 - 0.0625 = 0.0375 |
| 5 | 0.03125 | 1 | 0.0375 - 0.03125 = 0.00625 |
| 6 | 0.015625 | 0 | 0.00625 |
| 7 | 0.0078125 | 0 | 0.00625 |
| 8 | 0.00390625 | 1 | 0.00625 - 0.00390625 = 0.00234375 |
| 9 | 0.001953125 | 1 | 0.0234375 - 0.001953125 = 0.000390625 |
| 10 | 0.0009765625 | 0 | 0.000390625 |
| ⋮ | ⋮ | | |

# Digital Storage of Floating Point Numbers (8)

Calculations on the preceding slide show that
the binary mantissa for 0.1 is $(0001\,1001\,1\ldots)_2$.

**The decimal value of 0.1 cannot be represented by a finite number of binary digits.**

# Consequences of Finite Storage (1)

| | | |
|---|---|---|
| Limiting the number of bits allocated for storage of the exponent | $\Longrightarrow$ | Upper and lower limits on the **range** (or magnitude) of floating point numbers |
| Limiting the number of bits allocated for storage of the mantissa | $\Longrightarrow$ | Limit on the **precision** (or number of significant digits) for any floating point number. |

# Consequences of Finite Storage (2)

Most real numbers cannot be stored exactly (they do not exist on the floating point number line)

- Integers less than $2^{52}$ can be stored exactly.

  Try this:

```
>> x = 2^51
>> s = dec2bin(x)
>> x2 = bin2dec(s)
>> x2 - x
```

- Numbers with 15 (decimal) digit mantissas that are the exact sum of powers of (1/2) can be stored exactly.

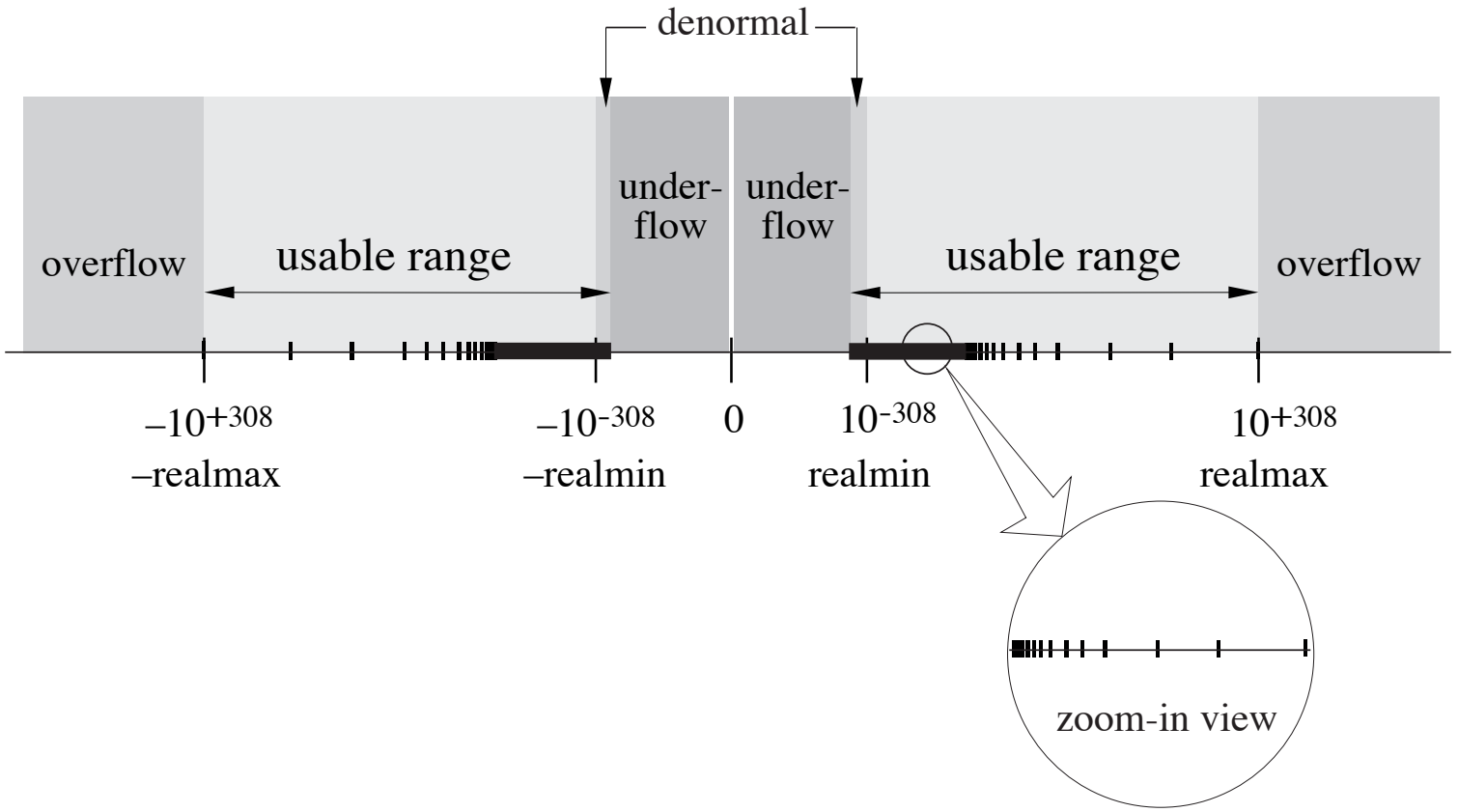
# Floating Point Number Line

Compare floating point numbers to real numbers.

| | Real numbers | Floating point numbers |
|---|---|---|
| **Range** | **Infinite**: arbitrarily large and arbitrarily small real numbers exist. | **Finite**: the number of bits allocated to the exponent limit the magnitude of floating point values. |
| **Precision** | **Infinite**: There is an infinite set of real numbers between any two real numbers. | **Finite**: there is a finite number (perhaps zero) of floating point values between any two floating point values. |

**In other words:** The floating point number line is a subset of the real number line.

# Floating Point Number Line



# Symbolic versus Numeric Calculation (1)

Commercial software for symbolic computation

- Derive$^{\mathrm{TM}}$
- MACSYMA$^{\mathrm{TM}}$
- Maple$^{\mathrm{TM}}$
- Mathematica$^{\mathrm{TM}}$

Symbolic calculations are exact. No rounding occurs because symbols and algebraic relationships are manipulated without storing numerical values.

# Symbolic versus Numeric Calculation (2)

**Example:** Evaluate $f(\theta) = 1 - \sin^2\theta - \cos^2\theta$

```
>> theta = 30*pi/180;
>> f = 1 - sin(theta)^2 - cos(theta)^2
f =
  -1.1102e-16
```

`f` is close to, but not exactly equal to zero because of *roundoff*. Also note that `f` is a single value, not a formula.

# Symbolic versus Numeric Calculation (3)

Symbolic computation using the Symbolic Math Toolbox in MATLAB

```
>> t = sym('t')     %  declare t as a symbolic variable
t =
t

>> f = 1 - sin(t)^2 - cos(t)^2  %  create a symbolic expression
f =
1-sin(t)^2-cos(t)^2

>> simplify(f)    %  ask Maple engine to make algebraic simplifications
f =
0
```

In the symbolic computation, `f` is exactly zero for any value of `t`. There is no roundoff error in symbolic computation.

# Numerical Arithmetic

Numerical values have limited range and precision. Values created by adding, subtracting, multiplying, or dividing floating point values will also have limited range and precision.

Quite often, the result of an arithmetic operation between two floating point values cannot be represented as another floating point value.

# Integer Arithmetic

| Operation | Result |
|---|---|
| $2 + 2 = 4$ | integer |
| $9 \times 7 = 63$ | integer |
| $\frac{12}{3} = 4$ | integer |
| $\frac{29}{13} = 2$ | exact result is not an integer |
| $\frac{29}{1300} = 0$ | exact result is not an integer |

# Floating Point Arithmetic

| Operation | Floating Point Value is . . . |
|---|---|
| $2.0 + 2.0 = 4$ | exact |
| $9.0 \times 7.0 = 63$ | exact |
| $\frac{12.0}{3.0} = 4$ | exact |
| $\frac{29}{13} = 2.2307692307692 31$ | approximate |
| $\frac{29}{1300} = 2.2307692307692 31 \times 10^{-2}$ | approximate |

# Floating Point Arithmetic in MATLAB (1)

```
>> format long e
>> u = 29/13
u =
     2.230769230769231e+00

>> v = 13*u
v =
    29

>> v-29
ans =
     0
```

Two rounding errors are made: (1) during computation and storage of `u`, and (2) during computation and storage of `v`. Fortuitously, the combination of rounding errors produces the exact result.

# Floating Point Arithmetic in MATLAB (2)

```
>> x = 29/1300
x =
     2.230769230769231e-02
>> y = 29 - 1300*x
y =
    3.552713678800501e-015
```

In exact arithmetic, the value of y should be zero.

The roundoff error occurs when x is stored. Since $29/1300$ cannot be expressed with a finite sum of the powers of $1/2$, the numerical value stored in x is a truncated approximation to $29/1300$.

When y is computed, the expression 1300*x evaluates to a number slightly different than 29 because the bits lost in the computation and storage of x are not recoverable.

# Catastrophic Cancellation Errors (1)

The errors in

$$c = a + b \qquad \text{and} \qquad c = a - b$$

will be large when $a \gg b$ or $a \ll b$.

Consider $c = a + b$ with

$$a = x.xxx \ldots \times 10^{0}$$
$$b = y.yyy \ldots \times 10^{-8}$$

where $x$ and $y$ are decimal digits.

# Catastrophic Cancellation Errors (1)

Evaluate $c = a + b$ with $a = x.xxx \ldots \times 10^0$ and $b = y.yyy \ldots \times 10^{-8}$

Assume for convenience of exposition that $z = x + y < 10$.

$$
\begin{array}{rl}
 & \overbrace{\texttt{x.xxx xxxx xxxx xxxx}}^{\text{available precision}} \\
+ & \texttt{0.000 0000 yyyy yyyy yyyy yyyy} \\
\hline
= & \texttt{x.xxx xxxx zzzz zzzz } \underbrace{\texttt{yyyy yyyy}}_{\text{lost digits}}
\end{array}
$$

The most significant digits of $a$ are retained, but the least significant digits of $b$ are lost because of the mismatch in magnitude of $a$ and $b$.

# Catastrophic Cancellation Errors (2)

**For subtraction**: The error in

$$c = a - b$$

will be large when $a \approx b$.

Consider $c = a - b$ with

$$a = x.xxxxxxxxxxx1ssssss$$
$$b = x.xxxxxxxxxxx0tttttt$$

where $x$, $y$, $s$ and $t$ are decimal digits. The digits $sss\ldots$ and $ttt\ldots$ are lost when $a$ and $b$ are stored in double-precision, floating point format.

# Catastrophic Cancellation Errors (3)

Evaluate $a - b$ in double precision floating point arithmetic when $a = x.xxx\,xxxx\,xxxx\,1$ and $b = x.xxx\,xxxx\,xxxx\,0$

$$
\begin{array}{rl}
 & \overbrace{\mathrm{x.xxx\,xxxx\,xxxx\,1}}^{\text{available precision}} \\
- & \mathrm{x.xxx\,xxxx\,xxxx\,0} \\
\hline
= & 0.000\,0000\,0000\,1\ \underbrace{\mathrm{uuuu\,uuuu\,uuuu}}_{\text{unassigned digits}} \\
 & \\
= & 1.\mathrm{uuuu\,uuuu\,uuuu} \times 10^{-12}
\end{array}
$$

The result has only one significant digit. Values for the uuuu digits are not necessarily zero. The *absolute* error in the result is small compared to either $a$ or $b$. The *relative* error in the result is large because $ssssss - ttttt \neq uuuuuu$ (except by chance).

# Catastrophic Cancellation Errors (4)

## Summary

- Occurs in addition $\alpha + \beta$ or subtraction $\alpha - \beta$ when $\alpha \gg \beta$ or $\alpha \ll \beta$
- Occurs in subtraction: $\alpha - \beta$ when $\alpha \approx \beta$
- Error caused by a single operation (hence the term "catastrophic") not a slow accumulation of errors.
- Can often be minimized by algebraic rearrangement of the troublesome formula. (Cf. improved quadratic formula.)

# Machine Precision (1)

The magnitude of roundoff errors is quantified by *machine precision* $\varepsilon_m$.

There is a number, $\varepsilon_m > 0$, such that

$$1 + \delta = 1$$

whenever $|\delta| < \varepsilon_m$.

In exact arithmetic $1 + \delta = 1$ only when $\delta = 0$, so in exact arithmetic $\varepsilon_m$ is identically zero.

MATLAB uses double precision (64 bit) arithmetic. The built-in variable **eps** stores the value of $\varepsilon_m$.

$$\texttt{eps} = 2.2204 \times 10^{-16}$$

# Machine Precision (2)

MATLAB code for Computing Machine Precision:

```
epsilon = 1;
it = 0;
maxit = 100;
while it < maxit
    epsilon = epsilon/2;
    b = 1 + epsilon;
    if b == 1
        break;
    end
    it = it + 1;
end
```

# Implications for Routine Calculations

- Floating point comparisons should test for "close enough" instead of exact equality.

- Express "close" in terms of

  absolute difference, $|x - y|$

  or

  relative difference, $\dfrac{|x - y|}{|x|}$

# Floating Point Comparison

Don't ask "is $x$ equal to $y$".

```
if x==y       %  Don't do this
   ...
end
```

Instead ask, "are $x$ and $y$ 'close enough' in value"

```
if abs(x-y) < tol
   ...
end
```

# Absolute and Relative Error (1)

"Close enough" can be measured with either absolute difference or relative difference, or both

Let

$$\alpha = \text{some exact or reference value}$$

$$\widehat{\alpha} = \text{some computed value}$$

*Absolute error*

$$E_{\text{abs}}(\widehat{\alpha}) = \left|\widehat{\alpha} - \alpha\right|$$

# Absolute and Relative Error (1)

*Relative error*

$$E_{\mathrm{rel}}(\widehat{\alpha}) = \frac{\left|\widehat{\alpha} - \alpha\right|}{\left|\alpha_{\mathrm{ref}}\right|}$$

Often we choose $\alpha_{\mathrm{ref}} = \alpha$ so that

$$E_{\mathrm{rel}}(\widehat{\alpha}) = \frac{\left|\widehat{\alpha} - \alpha\right|}{\left|\alpha\right|}$$

# Absolute and Relative Error (2)

**Example:** *Approximating* $\sin(x)$ *for small* $x$

Since

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

we can approximate $\sin(x)$ with

$$\sin(x) \approx x$$

for small enough $|x| < 1$

# Absolute and Relative Error (3)

The absolute error in approximating $\sin(x) \approx x$ for small $x$ is

$$E_{\text{abs}} = x - \sin(x) = \frac{x^3}{3!} - \frac{x^5}{5!} + \ldots$$

And the relative error is

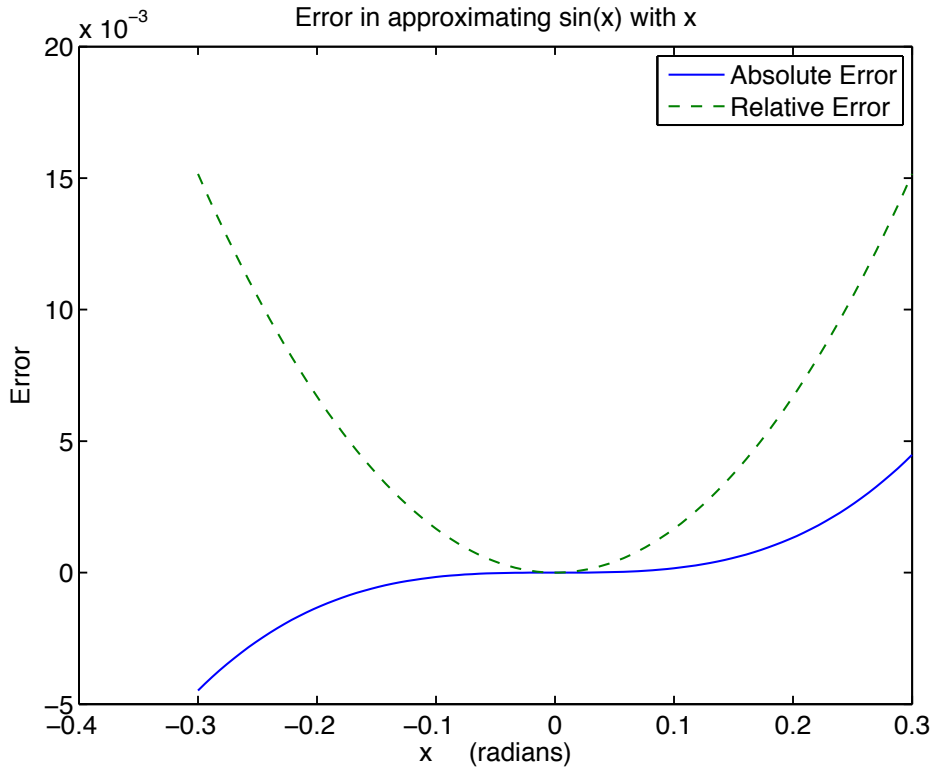
$$E_{\text{abs}} = \frac{x - \sin(x)}{\sin(x)} = \frac{x}{\sin(x)} - 1$$

# Absolute and Relative Error (4)

Plot relative and absolute error in approximating $\sin(x)$ with $x$.

Although the absolute error is relatively flat around $x = 0$, the relative error grows more quickly.

The relative error grows quickly because the absolute value of $\sin(x)$ is small near $x = 0$.



# Iteration termination (1)

An iteration generates a sequence of scalar values $x_k,\ k = 1, 2, 3, \ldots$.
The sequence converges to a limit $\xi$ if

$$|x_k - \xi| < \delta, \qquad \text{for all } k > N,$$

where $\delta$ is a small.

In practice, the test is usually expressed as

$$|x_{k+1} - x_k| < \delta, \quad \text{when } k > N.$$

# Iteration termination (2)

**Absolute** convergence criterion

Iterate until $|x - x_{\text{old}}| < \Delta_a$ where $\Delta_a$ is the absolute convergence tolerance.

In MATLAB:

```
x = ...                 %  initialize
xold = ...

while abs(x-xold) > deltaa
      xold = x;
      update x
end
```

# Iteration termination (3)

**Relative** convergence criterion

Iterate until $\left|\frac{x - x_{\text{old}}}{x_{\text{old}}}\right| < \delta_r$, where $\delta_r$ is the absolute convergence tolerance.

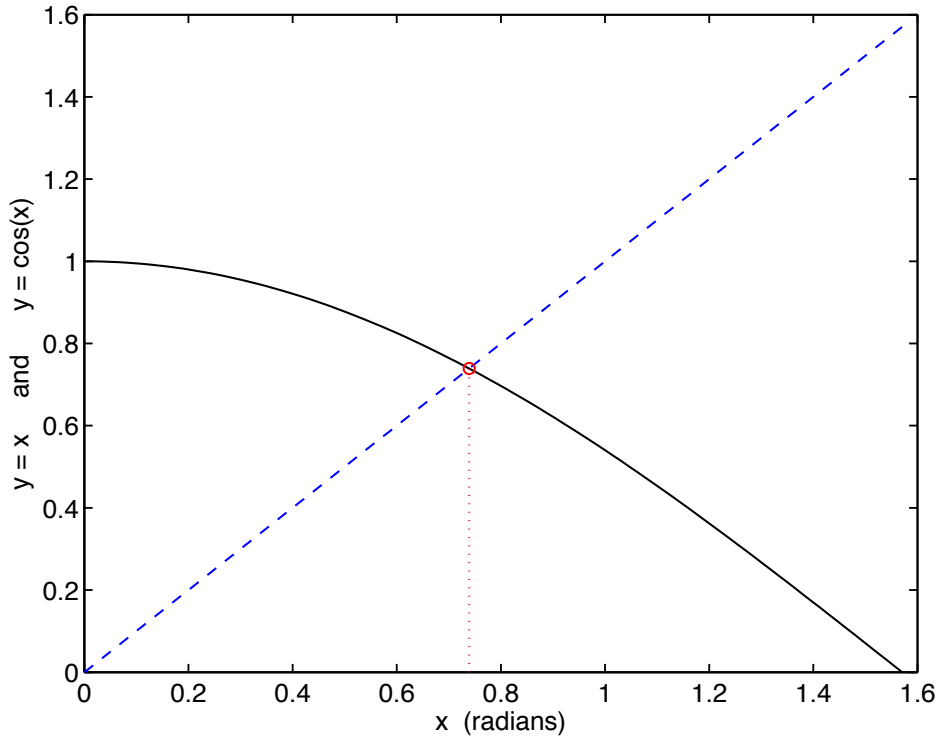
In MATLAB:

```
x = ...                  %  initialize
xold = ...

while abs((x-xold)/xold) > deltar
      xold = x;
      update x
end
```

# Example: Solve $\cos(x) = x$ (1)

Find the value of $x$ that satisfies $\cos(x) = x$.



## Example: Solve $\cos(x) = x$ (2)

The fixed point iteration as a method for obtaining a numerical approximation to the solution of a scalar equation. For now, trust that the follow algorithm will eventually give the solution.

1. Guess $x_0$

2. Set $x_{old} = x_0$

3. Update guess
$$x_{new} = \cos(x_{old})$$

4. If $x_{\text{new}} \approx x_{\text{old}}$ stop; otherwise set $x_{\text{old}} = x_{\text{new}}$ and return to step 3

## **Solve** $\cos(x) = x$ (3)

MATLAB implementation

```
x0 = ...       %  initial guess
k = 0;
xnew = x0;
while NOT_CONVERGED & k < maxit
    xold = xnew;
    xnew = cos(xold);
     k  =  k + 1;
end
```

Let's examine someways of defining the logical value `NOT_CONVERGED`.

## **Solve** $\cos(x) = x$ (4)

Bad test # 1

```
while xnew ~= xold
```

This test will be true unless `xnew` and `xold` are *exactly* equal. In other words, `xnew` and `xold` are equal only when their bit patterns are identical. This is bad because

- Test may never be met because of oscillatory bit patterns
- Even if test is eventually met, the iterations will probably do more work than needed

## **Solve** $\cos(x) = x$ (5)

Bad test # 2

```
while (xnew-xold) > delta
```

This test evaluates to false whenever `(xnew-xold)` is negative, even if $|(\texttt{xnew} - \texttt{xold})| \gg \texttt{delta}$.

**Example:**

```
>> xold = 100;  xnew = 1;  delta = 5e-9;
>> (xnew-xold) > delta
ans =
     0
```

These values of `xnew` and `xold` are not close, but the erroneous convergence criterion would cause the iterations to stop.

**Solve** $\cos(x) = x$ (6)

Workable test # 1: **Absolute tolerance**

```
while abs(xnew-xold) > delta
```

An absolute tolerance is useful when the iterative sequence converges to a value with magnitude much less than one.

What value of `delta` to use?

## **Solve** $\cos(x) = x$ (7)

Workable test # 2: **Relative tolerance**

```
while abs( (xnew-xold)/xref ) > delta
```

The user supplies appropriate value of `xref`. For this particular iteration we could use `xref` = `xold`.

```
while abs( (xnew-xold)/xold ) > delta
```

**Note:** For the problem of solving $\cos(x) = x$, the solution is $\mathcal{O}(1)$ so the absolute and relative convergence tolerance will terminate the calculations at roughly the same iteration.

## **Solve** $\cos(x) = x$ (8)

Using the relative convergence tolerance, the code becomes

```
x0 = ...       %  initial guess
k = 0;
xnew = x0;
while ( abs( (xnew-xold)/xold ) > delta )  &  k < maxit
    xold = xnew;
    xnew = cos(xold);
    k  = k  + 1;
end
```

**Note:** Parentheses around `abs( (xnew-xold)/xold ) > delta` are not needed for correct MATLAB implementation. The parenthesis are added to make the meaning of the clear to humans reading the code.

# Truncation Error

Consider the series for $\sin(x)$

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

For small $x$, only a few terms are needed to get a good approximation to $\sin(x)$. The $\cdots$ terms are "truncated"

$$f_{\text{true}} = f_{\text{sum}} + \text{truncation error}$$

The size of the truncation error depends on $x$ *and* the number of terms included in $f_{\text{sum}}$.

# Truncation of series for $\sin(x)$ (1)

```
function ssum = sinser(x,tol,n)
% sinser  Evaluate the series representation of the sine function
%
% Input:    x   = argument of the sine function, i.e., compute sin(x)
%           tol = tolerance on accumulated sum. Default:  tol = 5e-9
%           n   = maximum number of terms. Default: n = 15
%
% Output:   ssum = value of series sum after nterms or tolerance is met

term = x;  ssum = term;          %  Initialize series
fprintf('Series approximation to sin(%f)\n\n  k      term         ssum\n',x);
fprintf('%3d  %11.3e  %12.8f\n',1,term,ssum);
for k=3:2:(2*n-1)
  term = -term * x*x/(k*(k-1));                  %  Next term in the series
  ssum = ssum + term;
  fprintf('%3d  %11.3e  %12.8f\n',k,term,ssum);
  if abs(term/ssum)<tol, break;  end             %  True at convergence
end
fprintf('\nTruncation error after %d terms is %g\n\n',(k+1)/2,abs(ssum-sin(x)));
```

# Truncation of series for $\sin(x)$ (2)

For small $x$, the series for $\sin(x)$ converges in a few terms

```
>> s = sinser(pi/6,5e-9,15);
Series approximation to sin(0.523599)

  k       term          ssum
  1   5.236e-001    0.52359878
  3  -2.392e-002    0.49967418
  5   3.280e-004    0.50000213
  7  -2.141e-006    0.49999999
  9   8.151e-009    0.50000000
 11  -2.032e-011    0.50000000

Truncation error after 6 terms is 3.56382e-014
```

## Truncation of series for $\sin(x)$ (3)

The truncation error in the series is small relative to the true value of $\sin(\pi/6)$

```
>> s = sinser(pi/6,5e-9,15);
    :
>> err = (s-sin(pi/6))/sin(pi/6)
err =
 -7.1276e-014
```

# Truncation of series for $\sin(x)$ (4)

For larger $x$, the series for $\sin(x)$ converges more slowly

```
>> s = sinser(15*pi/6,5e-9,15);
Series approximation to sin(7.853982)

  k       term          ssum
  1    7.854e+000    7.85398163
  3   -8.075e+001  -72.89153055
  5    2.490e+002  176.14792646
  ⋮       ⋮             ⋮
 25    1.537e-003    1.00012542
 27   -1.350e-004    0.99999038
 29    1.026e-005    1.00000064

Truncation error after 15 terms is 6.42624e-007
```

Increasing the number of terms will allow the series to converge with the tolerance of $5 \times 10^{-9}$. A better solution to the slow convergence of the series are explored in Exercise 23.

# Taylor Series (1)

For a sufficiently continuous function $f(x)$ defined on the interval $x \in [a, b]$ we define the $n^{th}$ order Taylor Series approximation $P_n(x)$

$$P_n(x) = f(x_0) + (x - x_0) \left.\frac{df}{dx}\right|_{x=x_0} + \frac{(x - x_0)^2}{2} \left.\frac{d^2 f}{dx^2}\right|_{x=x_0} + \cdots + \frac{(x - x_0)^n}{n!} \left.\frac{d^n f}{dx^n}\right|_{x=x_0}$$

Then there exists $\xi$ with $x_0 \leq \xi \leq x$ such that

$$f(x) = P_n(x) + R_n(x)$$

where

$$R_n(x) = \frac{(x - x_0)^{(n+1)}}{(n+1)!} \left.\frac{d^{(n+1)} f}{dx^{(n+1)}}\right|_{x=\xi}$$

# Taylor Series (2)

Big "$\mathcal{O}$" notation

$$f(x) = P_n(x) + \mathcal{O}\left(\frac{(x - x_0)^{(n+1)}}{(n+1)!}\right)$$

or, for $x - x_0 = h$ we say

$$f(x) = P_n(x) + \mathcal{O}\left(h^{(n+1)}\right)$$

# Taylor Series Example

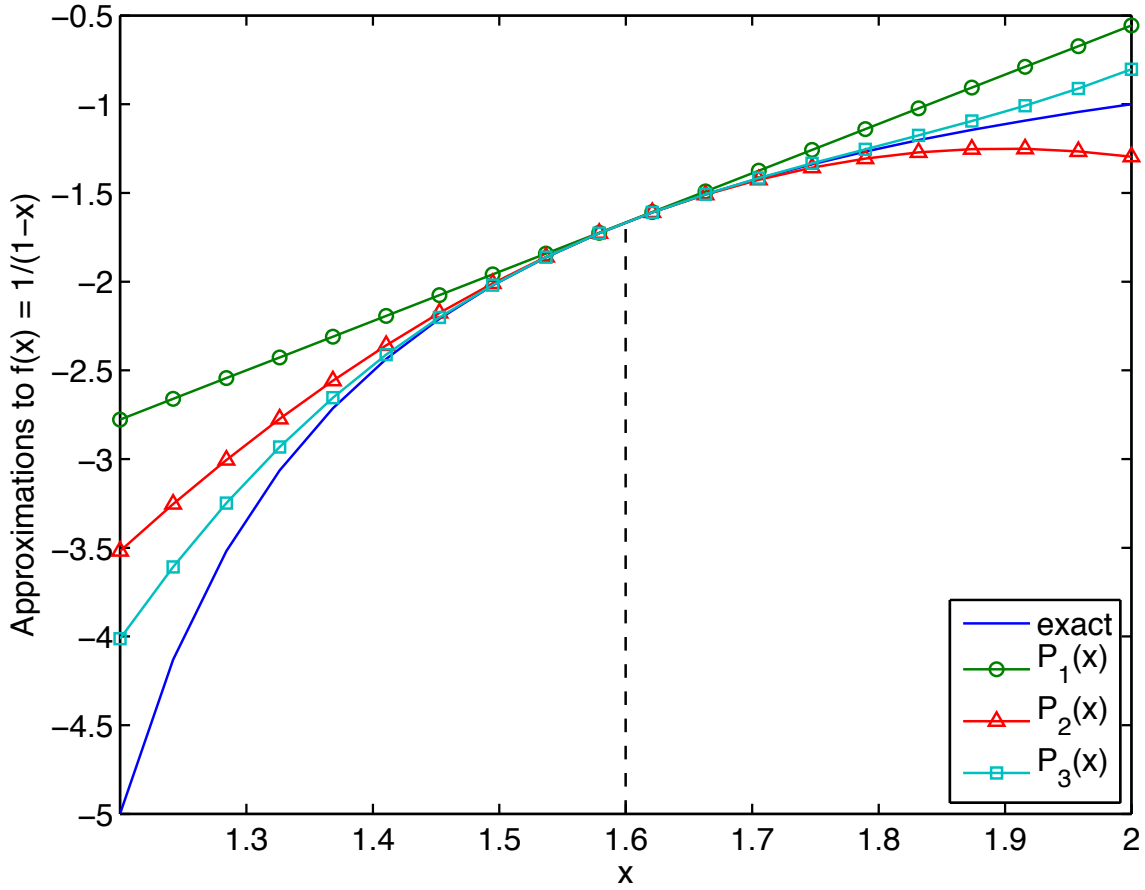
Consider the function

$$f(x) = \frac{1}{1-x}$$

The Taylor Series approximations to $f(x)$ of order 1, 2 and 3 are

$$\begin{aligned}
P_1(x) &= \frac{1}{1-x_0} + \frac{x-x_0}{(1-x_0)^2} \\
P_2(x) &= \frac{1}{1-x_0} + \frac{x-x_0}{(1-x_0)^2} + \frac{(x-x_0)^2}{(1-x_0)^3} \\
P_3(x) &= \frac{1}{1-x_0} + \frac{x-x_0}{(1-x_0)^2} + \frac{(x-x_0)^2}{(1-x_0)^3} + \frac{(x-x_0)^3}{(1-x_0)^4}
\end{aligned}$$

## Taylor Series: $P_i(x)$ near $x = 1.6$ (3)



# Roundoff and Truncation Errors (1)

Roundoff and truncation errors occur in numerical computation.

**Example:**

Finite difference approximation to $f'(x) = df/dx$

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f''(x) + \ldots$$

This approximation is said to be first order because the leading term in the truncation error is linear in $h$. Dropping the truncation error terms we obtain

$$f'_{fd}(x) = \frac{f(x+h) - f(x)}{h} \qquad \text{or} \qquad f'_{fd}(x) = f'(x) + \mathcal{O}(h)$$

# Roundoff and Truncation Errors (2)

To study the roles of roundoff and truncation errors, compute the finite difference[2] approximation to $f'(x)$ when $f(x) = e^x$.

The relative error in the $f'_{fd}(x)$ approximation to $\dfrac{d}{dx}e^x$ is

$$E_{\text{rel}} = \frac{f'_{fd}(x) - f'(x)}{f'(x)} = \frac{f'_{fd}(x) - e^x}{e^x}$$

[2]The finite difference approximation is used to obtain numerical solutions to ordinary and partial differentials equations where $f(x)$ and hence $f'(x)$ is *unknown*.

# Roundoff and Truncation Errors (3)

Evaluate $E_{\text{rel}}$ at $x = 1$ for a range of $h$.

Truncation error dominates at large $h$.

Roundoff error in $f(x+h) - f(h)$ dominates as $h \to 0$.

